1. Existing `game.py` and its problems

The current `game.py` has a `Mastermind` class with three existing attributes:

- `code` : the randomly generated four-digit secret.
- `history` : a list of guesses and their feedback.
- `turns` : the number of remaining guesses, initially 10.

The existing `run()` method already records valid guesses and checks for a win. However, it has no explicit game status.

The main lifecycle issues are:

Issue	Existing behaviour	Required improvement
Game status	No status attribute	Track `active` , `won` , `lost` and `quit` .
Winning	Prints a success message	Set status to `won` and display the result.
Losing	Prints the secret code	Set status to `lost` and display a clear message.
History	Stores accepted guesses	Preserve all accepted guesses throughout the game, including after termination.
Guess limit	Uses a `while self.turns` loop	Explicitly prevent further guesses once the limit is reached.
Final-turn win	Checks for a win before leaving the loop	Preserve this ordering and explicitly test it.
Quitting	Returns from `run()`	Set status to `quit` and return cleanly.

The existing class and its `__init__()` and `run()` methods can be retained. No additional dependencies or storage are needed.

2. Modified `game.py`

Replace the contents of `game.py` with the following. It retains the original class and method structure and changes only the lifecycle logic.

```
import random
from logic import feedback
```

```
class Mastermind:
    def __init__(self):
        self.code = [str(random.randint(1, 6)) for _ in range(4)]
        self.history = []
        self.turns = 10
        self.status = "active"

    def run(self):
        # Do not restart or continue a completed game.
        if self.status != "active":
            return

        print("Mastermind — enter four digits from 1 to 6.")

        while self.status == "active" and self.turns > 0:
            raw = input(f"{self.turns} turns left > ").strip()

            if raw.lower() == "q":
                self.status = "quit"
                print("Game quit.")
                return

            if len(raw) != 4 or any(ch not in "123456" for ch in raw):
                print("Enter exactly four digits from 1 to 6.")
                continue

            guess = list(raw)
            exact, partial = feedback(self.code, guess)

            self.history.append((raw, exact, partial))
            self.turns -= 1

            print("Exact:", exact, " Partial:", partial)

            # Check for a win before checking the guess limit.
            if exact == 4:
                self.status = "won"
                print("Cracked the code! You win!")
            elif self.turns == 0:
                self.status = "lost"
                print("Out of guesses! The code was",
                      "".join(self.code))
```

3. Explanation of modifications

A. Added explicit game status

`self.status` is initialized to `"active"`. The other possible values are `"won"`, `"lost"` and `"quit"`.

This allows the game to distinguish between a player successfully solving the code, exhausting their guesses and voluntarily quitting.

B. Preserved complete guess history

Every accepted guess is appended to `self.history` as:

Python Run

Invalid guesses and quitting do not add entries. The history is not cleared when the game ends, so it remains available for inspection.

C. Enforced the guess limit

The loop continues only while the game is active and guesses remain:

Python Run

Once the game ends, subsequent calls to `run()` return immediately. No additional guesses can be accepted.

D. Correctly handled the final permitted guess

The win condition is checked before the loss condition. If the player guesses the secret code on their tenth and final attempt, the status becomes `"won"`, not `"lost"`.

E. Cleanly handled quitting

Entering `q` or `Q` changes the status to `"quit"`, prints a message and exits `run()` immediately. Previously accepted guesses remain in the history.

4. Boundary tests

You can test the lifecycle using Python's built-in `unittest` module and `unittest.mock`. These tests do not require changing any project files.

Create a temporary file named `test_game.py` in the project directory.

```
import unittest
from unittest.mock import patch
from game import Mastermind


class TestMastermindLifecycle(unittest.TestCase):

    def create_game(self, code="1234", turns=10):
        game = Mastermind()
        game.code = list(code)
        game.turns = turns
        return game

    def test_win(self):
        game = self.create_game()

        with patch("builtins.input", side_effect=["1234"]):
            game.run()

        self.assertEqual(game.status, "won")
        self.assertEqual(game.turns, 9)
        self.assertEqual(len(game.history), 1)

    def test_final_turn_win(self):
        game = self.create_game(turns=1)

        with patch("builtins.input", side_effect=["1234"]):
            game.run()

        self.assertEqual(game.status, "won")
        self.assertEqual(game.turns, 0)
        self.assertEqual(len(game.history), 1)

    def test_loss(self):
        game = self.create_game(code="1234", turns=1)

        with patch("builtins.input", side_effect=["5555"]):
            game.run()

        self.assertEqual(game.status, "lost")
```

```
        self.assertEqual(game.turns, 0)
        self.assertEqual(len(game.history), 1)

    def test_quit(self):
        game = self.create_game()

        with patch("builtins.input", side_effect=["q"]):
            game.run()

        self.assertEqual(game.status, "quit")
        self.assertEqual(game.turns, 10)
        self.assertEqual(game.history, [])

    def test_uppercase_quit(self):
        game = self.create_game()

        with patch("builtins.input", side_effect=["Q"]):
            game.run()

        self.assertEqual(game.status, "quit")

    def test_history_preserved(self):
        game = self.create_game(code="1234")

        with patch("builtins.input",
                   side_effect=["1111", "2222", "q"]):
            game.run()

        self.assertEqual(game.status, "quit")
        self.assertEqual(len(game.history), 2)
        self.assertEqual(game.history[0][0], "1111")
        self.assertEqual(game.history[1][0], "2222")

    def test_invalid_guess_does_not_consume_turn(self):
        game = self.create_game()

        with patch("builtins.input",
                   side_effect=["12", "1237", "1234"]):
            game.run()

        self.assertEqual(game.status, "won")
        self.assertEqual(game.turns, 9)
        self.assertEqual(len(game.history), 1)

    def test_no_guesses_after_win(self):
        game = self.create_game()

        with patch("builtins.input", side_effect=["1234"]):
            game.run()
```

```
        history = game.history.copy()
        turns = game.turns

        # Calling run again must not request more input.
        with patch("builtins.input",
                   side_effect=AssertionError("Unexpected input")):
            game.run()

        self.assertEqual(game.status, "won")
        self.assertEqual(game.history, history)
        self.assertEqual(game.turns, turns)

    def test_no_guesses_after_loss(self):
        game = self.create_game(turns=1)

        with patch("builtins.input", side_effect=["5555"]):
            game.run()

        with patch("builtins.input",
                   side_effect=AssertionError("Unexpected input")):
            game.run()

        self.assertEqual(game.status, "lost")
        self.assertEqual(len(game.history), 1)

    def test_no_guesses_after_quit(self):
        game = self.create_game()

        with patch("builtins.input", side_effect=["q"]):
            game.run()

        with patch("builtins.input",
                   side_effect=AssertionError("Unexpected input")):
            game.run()

        self.assertEqual(game.status, "quit")
        self.assertEqual(game.history, [])


if __name__ == "__main__":
    unittest.main()
```

Run the tests from your project directory:

Bash

Expected result

```
Ran 10 tests
OK
```

This is the expected output if all tests pass; they have not been run against your local repository.

5. Boundary cases to verify manually

Scenario	Expected status	Expected remaining turns
Correct guess on first attempt	`won`	9
Correct guess on final attempt	`won`	0
Incorrect guess on final attempt	`lost`	0
Player enters `q`	`quit`	Unchanged
Player enters an invalid guess	`active`	Unchanged
Call `run()` after winning	`won`	Unchanged
Call `run()` after losing	`lost`	Unchanged
Call `run()` after quitting	`quit`	Unchanged

Implementation summary

- Only `game.py` is modified.
- The original `Mastermind` class and its two methods are preserved.
- Game state and history remain in memory.
- No external dependencies or persistent storage are introduced.
- The final-guess win and post-termination cases are explicitly covered by tests.

One important distinction: the code above is the proposed replacement, not a confirmed edit to your GitHub repository. Apply it to your local `game.py`, then run the tests to verify it in your environment.

Extend the existing Mastermind game to support Easy, Medium and Hard difficulty modes.

Use these configurations:
- Easy: 3 positions, symbols 1–4, 12 guesses.
- Medium: 4 positions, symbols 1–6, 10 guesses.
- Hard: 5 positions, symbols 1–8, 8 guesses.

Allow the player to choose a difficulty when starting the game. Generate the secret code according to the selected difficulty. Update input validation, win detection, feedback

Show more

I'll extend the existing `Mastermind` design so that difficulty determines the code length, available symbols and maximum guesses. The duplicate-aware `feedback()` function will remain reusable across all three modes.

Difficulty configurations

Difficulty	Positions	Symbols	Guesses
Easy	3	1–4	12
Medium	4	1–6	10
Hard	5	1–8	8

1. Changes to the existing project

The repository currently has three separate files for the game: `main.py`, `game.py` and `logic.py`. The existing `game.py` hardcodes four positions, symbols `1–6` and ten guesses. `main.py` simply starts the game using `Mastermind().run()`. GitHub +1

The changes can be confined to `game.py` and `logic.py`. `main.py` can remain unchanged.

Design

- A single configuration dictionary holds the three difficulty settings.
- The player selects a difficulty when creating a game.
- The selected configuration controls code generation, input validation, feedback and the guess limit.
- The existing lifecycle states and in-memory guess history are retained.
- The feedback function handles repeated symbols and variable code lengths.

2. Updated `game.py`

```
import random
from logic import feedback


class Mastermind:
    DIFFICULTIES = {
        "easy": {
            "positions": 3,
            "symbols": "1234",
            "guesses": 12
        },
        "medium": {
            "positions": 4,
            "symbols": "123456",
            "guesses": 10
        },
        "hard": {
            "positions": 5,
            "symbols": "12345678",
            "guesses": 8
        }
    }

    def __init__(self, difficulty=None):
        if difficulty is None:
            while True:
                difficulty = input(
                    "Choose difficulty (Easy/Medium/Hard): "
                ).strip().lower()

                if difficulty in self.DIFFICULTIES:
                    break
```

```
                print("Invalid difficulty. Choose Easy, Medium or Hard.")

        difficulty = difficulty.strip().lower()
        if difficulty not in self.DIFFICULTIES:
            raise ValueError(
                "Difficulty must be Easy, Medium or Hard."
            )

        self.difficulty = difficulty
        self.config = self.DIFFICULTIES[difficulty]
        self.positions = self.config["positions"]
        self.symbols = self.config["symbols"]
        self.max_guesses = self.config["guesses"]

        self.code = [
            random.choice(self.symbols)
            for _ in range(self.positions)
        ]
        self.history = []
        self.turns = self.max_guesses
        self.status = "active"

    def run(self):
        if self.status != "active":
            return

        print(f"\nMastermind - {self.difficulty.title()} mode")
        print(
            f"Enter {self.positions} digits from "
            f"{self.symbols[0]} to {self.symbols[-1]}."
        )
        print("Enter 'q' to quit.")

        while self.status == "active" and self.turns > 0:
            raw = input(f"{self.turns} guesses left > ").strip()

            if raw.lower() == "q":
                self.status = "quit"
                print("Game quit.")
                return

            if (
                len(raw) != self.positions
                or any(ch not in self.symbols for ch in raw)
            ):
                print(
                    f"Enter exactly {self.positions} digits "
                    f"from {self.symbols[0]} to {self.symbols[-1]}."
```

```
                )
                continue

            guess = list(raw)
            exact, partial = feedback(self.code, guess)

            self.history.append((raw, exact, partial))
            self.turns -= 1

            print("Exact:", exact, " Partial:", partial)

            # Check for a win before checking the guess limit.
            if exact == self.positions:
                self.status = "won"
                print("Cracked the code! You win!")
            elif self.turns == 0:
                self.status = "lost"
                print("Out of guesses! The code was",
                      "".join(self.code))
```

3. Updated `logic.py`

The existing feedback function in the repository still uses membership checks, which can overcount repeated symbols. GitHub Replace it with the following duplicate-aware implementation. It works with three-, four- and five-position codes without needing to know the selected difficulty.

```
def feedback(code, guess):
    exact = 0
    partial = 0

    code_used = [False] * len(code)
    guess_used = [False] * len(guess)

    # First pass: count exact matches.
    for i in range(min(len(code), len(guess))):
        if code[i] == guess[i]:
            exact += 1
            code_used[i] = True
            guess_used[i] = True

    # Second pass: count partial matches.
    for i in range(len(guess)):
        if guess_used[i]:
```

```
            continue

        for j in range(len(code)):
            if not code_used[j] and guess[i] == code[j]:
                partial += 1
                code_used[j] = True
                guess_used[i] = True
                break

    return exact, partial
```

The function retains its original signature and `(exact, partial)` return format. Exact matches are counted first, and each symbol occurrence can be used at most once.

4. How difficulty affects gameplay

Easy 3 positions	**Example secret:** `142` The player must enter exactly three digits, each between 1 and 4. The game permits 12 valid guesses.
Medium 4 positions	**Example secret:** `5261` The player must enter exactly four digits, each between 1 and 6. The game permits 10 valid guesses.
Hard 5 positions	**Example secret:** `81736` The player must enter exactly five digits, each between 1 and 8. The game permits 8 valid guesses.

Repeated symbols are permitted in all three difficulties. For example, an Easy code can be `112`, and the feedback function will correctly handle guesses containing repeated `1`s.

5. Tests for all three difficulties

Create a temporary `test_difficulty.py` file in the project directory.

The following tests cover configuration, code generation, input validation, winning, losing and the final permitted guess.

```
import unittest
from unittest.mock import patch
from game import Mastermind


class TestDifficulty(unittest.TestCase):

    def test_all_difficulty_configurations(self):
        expected = {
            "easy": (3, "1234", 12),
            "medium": (4, "123456", 10),
            "hard": (5, "12345678", 8)
        }

        for difficulty, (positions, symbols, guesses) in expected.items():
            with self.subTest(difficulty=difficulty):
                game = Mastermind(difficulty)

                self.assertEqual(game.positions, positions)
                self.assertEqual(game.symbols, symbols)
                self.assertEqual(game.max_guesses, guesses)
                self.assertEqual(game.turns, guesses)
                self.assertEqual(game.status, "active")
                self.assertEqual(len(game.code), positions)
                self.assertTrue(all(ch in symbols for ch in game.code))

    def test_invalid_difficulty(self):
        with self.assertRaises(ValueError):
            Mastermind("extreme")

    def test_win_in_every_difficulty(self):
        configurations = {
            "easy": "123",
            "medium": "1234",
            "hard": "12345"
        }

        for difficulty, code in configurations.items():
            with self.subTest(difficulty=difficulty):
                game = Mastermind(difficulty)
                game.code = list(code)

                with patch("builtins.input", side_effect=[code]):
                    game.run()

                self.assertEqual(game.status, "won")
                self.assertEqual(len(game.history), 1)
                self.assertEqual(
```

```
                game.history[0],
                (code, len(code), 0)
            )

    def test_final_guess_win_in_every_difficulty(self):
        configurations = {
            "easy": ("123", 12),
            "medium": ("1234", 10),
            "hard": ("12345", 8)
        }

        for difficulty, (code, limit) in configurations.items():
            with self.subTest(difficulty=difficulty):
                game = Mastermind(difficulty)
                game.code = list(code)
                game.turns = 1

                with patch("builtins.input", side_effect=[code]):
                    game.run()

                self.assertEqual(game.status, "won")
                self.assertEqual(game.turns, 0)

    def test_loss_in_every_difficulty(self):
        configurations = {
            "easy": ("123", "444", 12),
            "medium": ("1234", "5555", 10),
            "hard": ("12345", "88888", 8)
        }

        for difficulty, (code, wrong, limit) in configurations.items():
            with self.subTest(difficulty=difficulty):
                game = Mastermind(difficulty)
                game.code = list(code)
                game.turns = 1

                with patch("builtins.input", side_effect=[wrong]):
                    game.run()

                self.assertEqual(game.status, "lost")
                self.assertEqual(game.turns, 0)
                self.assertEqual(len(game.history), 1)

    def test_invalid_input_does_not_consume_turn(self):
        game = Mastermind("easy")
        game.code = list("123")

        with patch(
            "builtins.input",
```

```
            side_effect=["12", "12345", "567", "123"]
        ):
            game.run()

        self.assertEqual(game.status, "won")
        self.assertEqual(game.turns, 11)
        self.assertEqual(len(game.history), 1)

    def test_quit_in_every_difficulty(self):
        for difficulty in ("easy", "medium", "hard"):
            with self.subTest(difficulty=difficulty):
                game = Mastermind(difficulty)

                with patch("builtins.input", side_effect=["q"]):
                    game.run()

                self.assertEqual(game.status, "quit")
                self.assertEqual(game.turns, game.max_guesses)
                self.assertEqual(game.history, [])

    def test_repeated_symbols_in_all_difficulties(self):
        cases = [
            ("easy", "112", "121", (1, 2)),
            ("medium", "1123", "2113", (2, 2)),
            ("hard", "11234", "21134", (3, 2))
        ]

        for difficulty, code, guess, expected in cases:
            with self.subTest(difficulty=difficulty):
                game = Mastermind(difficulty)
                game.code = list(code)

                self.assertEqual(
                    feedback(game.code, list(guess)),
                    expected
                )


if __name__ == "__main__":
    unittest.main()
```

Run the tests using:

```
Bash
```

Expected test coverage

8 test methods, including subtests for every difficulty.

The tests should verify that all three modes use the correct configuration and that the game handles the key boundary cases. They are provided for execution; they have not been run against your local files.

6. Files to modify

File	Action	
`game.py`	Modify	Add difficulty configuration, selection and dynamic gameplay.
`logic.py`	Modify	Use duplicate-aware feedback for variable code lengths.
`main.py`	Unchanged	The existing entry point works with the updated constructor.
`requirements.txt`	Unchanged	No new dependencies are required.

You can launch the game as before:

Bash

The player will now be prompted to choose a difficulty before the game begins.

Final result

The game now supports Easy, Medium and Hard while retaining the original modular structure, in-memory history, lifecycle management and duplicate-aware feedback. Only the two existing implementation files need to be updated.

Improve the existing game's guess history and input handling.

- Display a readable history of accepted guesses, including exact and partial feedback.

- Reject guesses with an incorrect length, unsupported symbols or malformed input.
- Invalid guesses must not consume turns or appear in the accepted-guess history.
- Ensure that feedback is calculated and recorded exactly once for each accepted guess.

Show more

The changes can be kept within `game.py`, leaving `main.py` and the overall game structure unchanged. I'll retain the Easy, Medium and Hard configurations, the existing quit command and the `status`-based game lifecycle.

The key improvement is to maintain a history containing **only accepted guesses**, with feedback calculated once and reused for both the history and the displayed result.

1. Updated `game.py`

Replace the existing `game.py` with the following:

```
import random
from logic import feedback


class Mastermind:
    DIFFICULTIES = {
        "easy": {"positions": 3, "symbols": "1234", "guesses": 12},
        "medium": {"positions": 4, "symbols": "123456", "guesses": 10},
        "hard": {"positions": 5, "symbols": "12345678", "guesses": 8}
    }

    def __init__(self, difficulty=None):
        if difficulty is None:
            while True:
                difficulty = input(
                    "Choose difficulty (Easy/Medium/Hard): "
                ).strip().lower()
                if difficulty in self.DIFFICULTIES:
                    break
                print("Invalid difficulty. Choose Easy, Medium or Hard.")
```

```
        difficulty = difficulty.strip().lower()
        if difficulty not in self.DIFFICULTIES:
            raise ValueError("Difficulty must be Easy, Medium or Hard.")

        self.difficulty = difficulty
        self.config = self.DIFFICULTIES[difficulty]
        self.positions = self.config["positions"]
        self.symbols = self.config["symbols"]
        self.max_guesses = self.config["guesses"]

        self.code = [
            random.choice(self.symbols)
            for _ in range(self.positions)
        ]
        self.history = []
        self.turns = self.max_guesses
        self.status = "active"

    def display_history(self):
        """Display only accepted guesses and their feedback."""
        if not self.history:
            print("No accepted guesses yet.")
            return

        print("\nGuess History")
        print("-" * 35)
        print(f"{'Turn':<8}{'Guess':<12}{'Exact':<8}{'Partial'}")
        print("-" * 35)

        for turn, (guess, exact, partial) in enumerate(
            self.history, start=1
        ):
            print(f"{turn:<8}{guess:<12}{exact:<8}{partial}")

        print("-" * 35)

    def run(self):
        if self.status != "active":
            return

        print(f"\nMastermind - {self.difficulty.title()} mode")
        print(
            f"Enter {self.positions} digits from "
            f"{self.symbols[0]} to {self.symbols[-1]}."
        )
        print("Enter 'q' to quit.")

        while self.status == "active" and self.turns > 0:
            raw = input(f"{self.turns} guesses left > ").strip()
```

```
        if raw.lower() == "q":
            self.status = "quit"
            print("Game quit.")
            self.display_history()
            return

        if (
            len(raw) != self.positions
            or any(ch not in self.symbols for ch in raw)
        ):
            print(
                f"Invalid guess. Enter exactly {self.positions} "
                f"digits from {self.symbols[0]} "
                f"to {self.symbols[-1]}."
            )
            continue

        # Calculate feedback exactly once per accepted guess.
        exact, partial = feedback(self.code, list(raw))

        self.history.append((raw, exact, partial))
        self.turns -= 1

        print(f"Exact: {exact} | Partial: {partial}")
        self.display_history()

        if exact == self.positions:
            self.status = "won"
            print("Cracked the code! You win!")
        elif self.turns == 0:
            self.status = "lost"
            print("Out of guesses! The code was",
                  "".join(self.code))
```

2. Tests

Create a file named `test_game.py` in the same directory as `game.py`. These tests use Python's built-in `unittest` framework, so no additional dependencies are needed.

```
import unittest
from unittest.mock import patch, call
from game import Mastermind
```

```
class TestGameHistoryAndInput(unittest.TestCase):

    def test_valid_guess_is_recorded(self):
        game = Mastermind("easy")
        game.code = list("123")

        with patch("builtins.input", side_effect=["123"]):
            with patch("game.feedback", wraps=__import__(
                "game"
            ).feedback) as mock_feedback:
                game.run()

        self.assertEqual(game.status, "won")
        self.assertEqual(game.history, [("123", 3, 0)])
        self.assertEqual(game.turns, 11)
        self.assertEqual(mock_feedback.call_count, 1)

    def test_invalid_guesses_do_not_consume_turns(self):
        game = Mastermind("easy")
        game.code = list("123")

        invalid_inputs = [
            "12",       # Too short
            "1234",     # Too long
            "567",      # Unsupported symbols
            "1a3",      # Non-numeric symbol
            "",         # Empty input
            "1 3",      # Embedded whitespace
            "１２３",   # Non-ASCII digits
        ]

        with patch(
            "builtins.input",
            side_effect=invalid_inputs + ["123"]
        ):
            game.run()

        self.assertEqual(game.status, "won")
        self.assertEqual(game.turns, 11)
        self.assertEqual(len(game.history), 1)
        self.assertEqual(game.history[0], ("123", 3, 0))

    def test_feedback_is_calculated_once_per_valid_guess(self):
        game = Mastermind("easy")
        game.code = list("123")

        with patch("game.feedback", return_value=(1, 2)) as mock_feedback:
            with patch(
```

```
                "builtins.input",
                side_effect=["12", "111", "321", "q"]
            ):
                game.run()

        self.assertEqual(mock_feedback.call_count, 2)
        self.assertEqual(
            mock_feedback.call_args_list,
            [call(list("111"), list("123"))]
        )

    def test_history_contains_only_accepted_guesses(self):
        game = Mastermind("medium")
        game.code = list("1234")

        with patch(
            "builtins.input",
            side_effect=["12", "12345", "1235", "1234"]
        ):
            game.run()

        self.assertEqual(
            game.history,
            [("1235", 3, 0), ("1234", 4, 0)]
        )
        self.assertEqual(game.status, "won")
        self.assertEqual(game.turns, 8)

    def test_quit_preserves_history_and_turns(self):
        game = Mastermind("easy")
        game.code = list("123")

        with patch(
            "builtins.input",
            side_effect=["111", "q"]
        ):
            game.run()

        self.assertEqual(game.status, "quit")
        self.assertEqual(len(game.history), 1)
        self.assertEqual(game.turns, 11)

    def test_ended_game_cannot_be_run_again(self):
        game = Mastermind("easy")
        game.code = list("123")

        with patch("builtins.input", side_effect=["123"]):
            game.run()
```

```
        history_before = list(game.history)
        turns_before = game.turns

        with patch("builtins.input") as mock_input:
            game.run()

        mock_input.assert_not_called()
        self.assertEqual(game.history, history_before)
        self.assertEqual(game.turns, turns_before)

    def test_history_display(self):
        game = Mastermind("easy")
        game.history = [
            ("112", 1, 2),
            ("123", 3, 0)
        ]

        with patch("builtins.print") as mock_print:
            game.display_history()

        output = "\n".join(
            str(args[0])
            for args, _ in mock_print.call_args_list
            if args
        )
        self.assertIn("Guess History", output)
        self.assertIn("112", output)
        self.assertIn("123", output)
        self.assertIn("Partial", output)


if __name__ == "__main__":
    unittest.main()
```

There is one correction needed in the feedback call-count test above: `feedback` receives `(code, guess)`, so the expected calls should be:

```
[
    call(list("123"), list("111")),
    call(list("123"), list("321"))
```

Python Run

3. Correct the feedback call-count assertion

In `test_feedback_is_calculated_once_per_valid_guess`, replace the `assertEqual` for `mock_feedback.call_args_list` with:

```python
self.assertEqual(
    mock_feedback.call_args_list,
    [
        call(list("123"), list("111")),
        call(list("123"), list("321"))
    ]
```

Python Run

This verifies that feedback is called once for each of the two accepted guesses and never for the invalid guess `12`.

4. What has changed?

Change	Behaviour
Readable history	Displays each accepted guess, its exact matches and partial matches.
Input validation	Rejects incorrect lengths, unsupported characters, empty input and embedded whitespace.
Turn management	Invalid guesses do not reduce the remaining turns.
History management	Invalid guesses are never recorded.
Feedback calculation	Called exactly once per accepted guess; the result is reused.
Quit command	`q` still exits without consuming a turn.
Lifecycle	Won, lost and quit games cannot be restarted using the same instance.

Expected history display

```
Guess History
-----------------------------------
Turn    Guess       Exact   Partial
-----------------------------------
1       112         1       2
```

```
2       123         3       0
-----------------------------------
```

5. Run the tests

From the directory containing `game.py` and `test_game.py`, run:

Bash

```
python -m unittest -v test_game.py
```

The tests cover valid guesses, invalid input, history, feedback call counts, quitting and the game lifecycle. They are provided for you to run; I haven't verified them against your local repository.

No changes to `logic.py` or `main.py` are necessary, provided your existing `feedback(code, guess)` function correctly handles duplicate symbols.

ChatGPT can make mistakes. Check important info.